

A* Algorithm

Het A* algoritme is een pathfinding algoritme die het kortste pad tussen een startende node en een eind node in een gewogen graaf

Het A* algoritme gebruikt de 2 beste aspecten van de volgende algoritmes:

- Dijkstra's algoritme: Zoekt het kortste pad naar alle nodes van 1 source node
- Greedy best-first search: Dit algoritme bekijkt de node die het dichtbijzijnde is naar het doel

Als je het korste pad zoekt tussen 2 steden op een map. Dijkstra's algoritme gaat dan in alle richtingen opzoeken en best-first search gaat misschien recht naar de bestemming en mist dan potentiële shortcuts. A* doet beide:

- De afstand die die af heeft gelegd vanaf de start
- Slimme afweging naar de afstand die nog afgelegd moet worden.

Deze combinatie helpt het a* algoritme om geïnformeerde beslissingen te maken over welk pad die het best als volgende kan bekijken, hierdoor is het erg efficiënt en accuraat.

Belangrijke begrippen

Om het a* algoritme te begrijpen moet je een aantal begrippen kennen:

- **Nodes:** Punten in je map (zoals kruispunten op een map)
- **Edges:** Connecties tussen nodes (Wegen die kruispunten connecten)
- **Path costs:** De echte waarde van het bewegen van de ene naar de andere node
- **Heuristic:** Afgewogen waarde van elke node naar het einde
- **Search space:** Collectie van alle mogelijke paden om te onderzoeken

Formule

Dit is de formule die gebruikt wordt voor het a* algoritme $f(n) = g(n) + h(n)$

Stel $g = 4$ en $h = 3$, dan is $f = 7$, A* kiest dan deze node boven een node met $f = 9$

Wat betekenen ze

$g(n)$ (de geschiedenis): Dit is de werkelijke kost om van het startpunt naar de huidige node te komen. Dit is het gedeelte dat van het dijkstra geleend is. Het zorgt ervoor dat we niet zomaar een pad kiezen dat er kort uitziet, maar dat we echt daadwerkelijk weten wat het kost. **$h(n)$** (De toekomst): Dit is de **Heuristic**. Het is een schatting van de afstand van de huidige node naar het einddoel. Dit is het Greedy best-first search aspect. **$f(n)$** (De totaalscore): A* kiest altijd de node met de laagste $f(n)$ waarde om als volgende te verkennen

Waarom is Heuristic zo belangrijk?

De kracht van A* valt of staat met heuristic(h):

- **Toelaatbaar:** Als h nooit hoger is dan de werkelijke afstand (als die de afstand korter inschat dan het daadwerkelijk is) dan zal a* gegarandeerd het kortste pad vinden.

- **Perfect:** Als h exact gelijk is aan de werkelijke afstand, loopt A* in een rechte lijn naar het eindpunt
- **Overschat:** Als h de afstand overschat, verliest A* zijn garantie op het kortste pad vinden en gaat het meer lijken op een zoektocht.

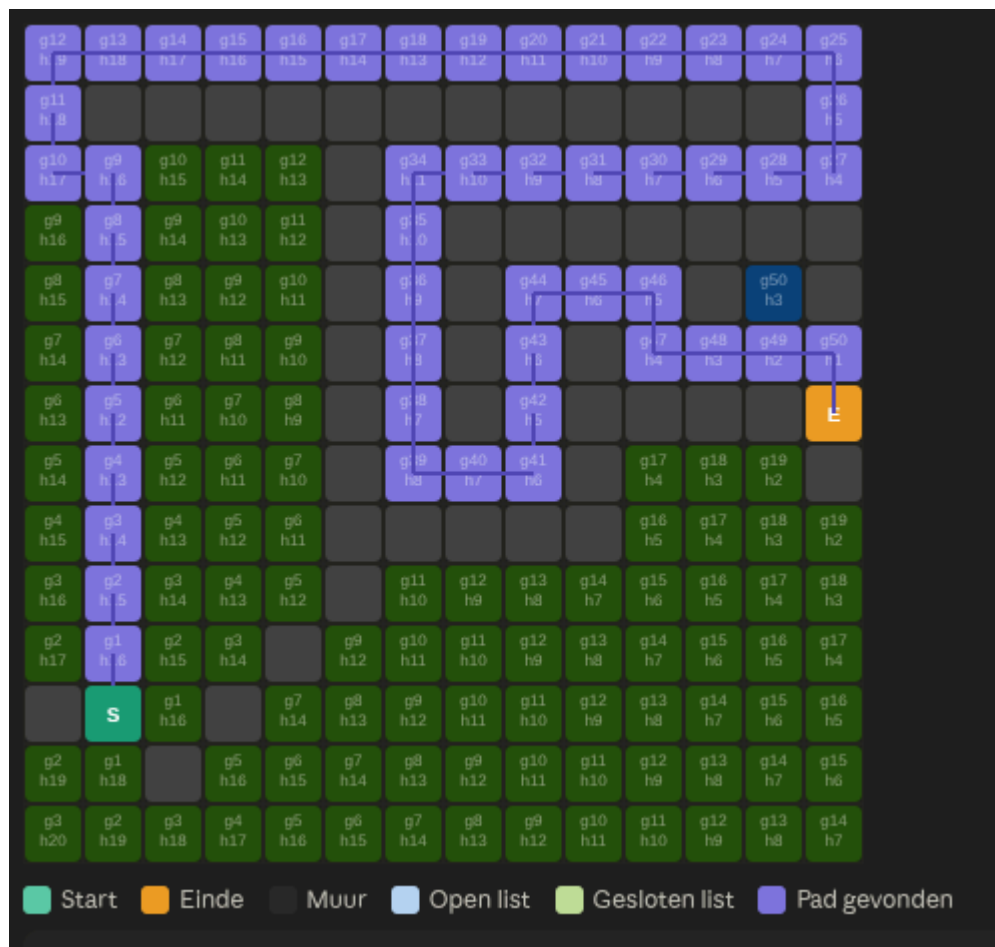
Admissible heuristic → overschat nooit

Consistent heuristic → zorgt ervoor dat nodes niet steeds opnieuw onderzocht hoeven worden

Simulatie gemaakt door Claude

Worst case

Ik heb een worst case gemaakt waarbij die alle vakjes behalve 1 langs moet voordat die bij het eindpunt komt. Dat ziet er zo uit.



Zonder muren

The grid map shows a search space with a start cell (green) at (row 4, col 2) and an end cell (orange) at (row 4, col 12). The path found is highlighted in yellow, passing through cells (row 4, col 3), (row 4, col 4), (row 4, col 5), (row 4, col 6), (row 4, col 7), (row 4, col 8), (row 4, col 9), (row 4, col 10), and (row 4, col 11). The legend indicates that the path consists of cells in the 'Open list' (light blue) and 'Gesloten list' (dark blue) states.

Met muren in het midden



Nog een redelijk trage opstelling

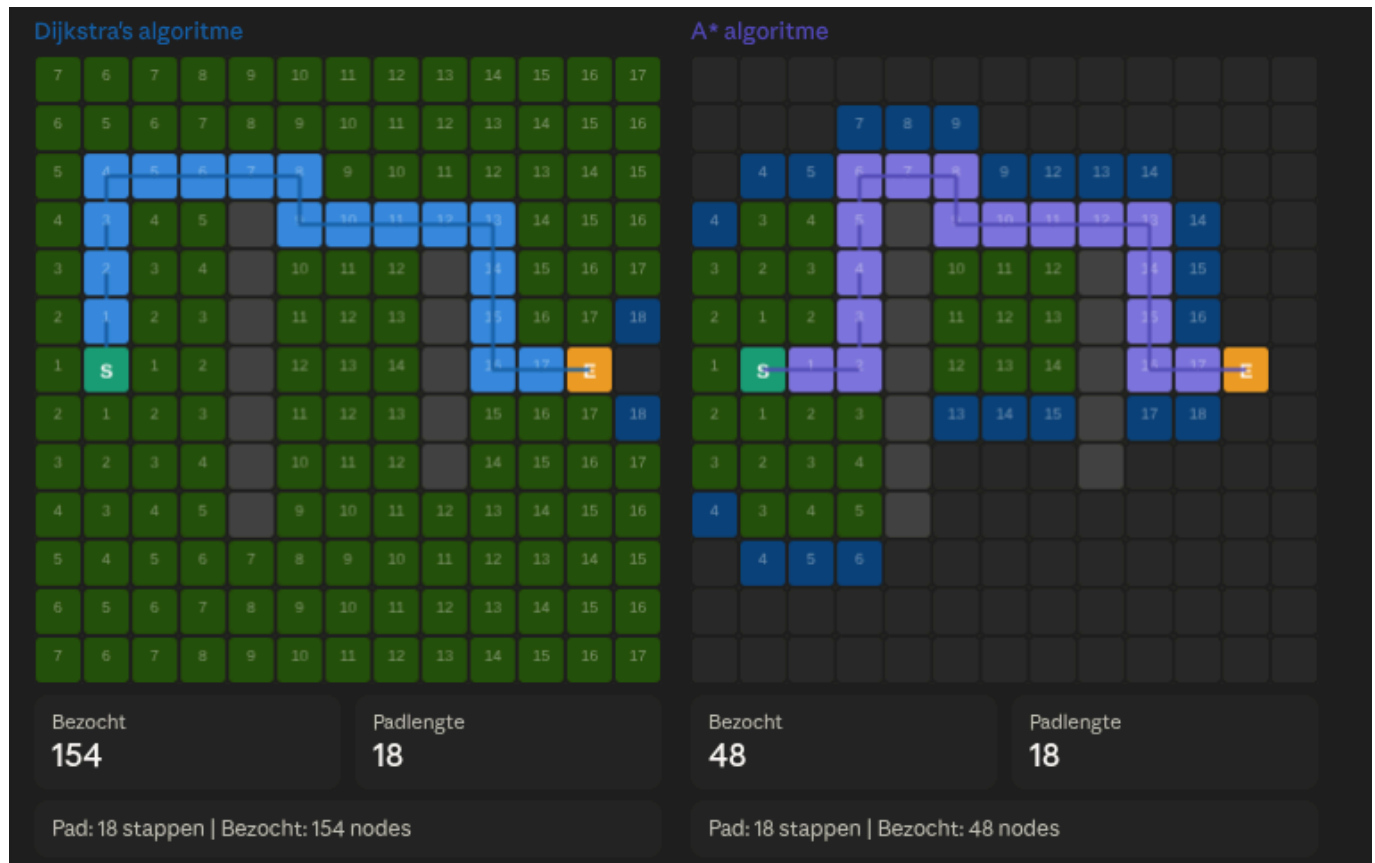
Ik heb nu aan de bovenkant hem iets langer gemaakt zodat ik hem via onder forceer en dan aan de onderkant het pad dicht gemaakt zodat die er langer over doet



A* vs Dijkstra

Ik heb aan Claude gevraagd om zoiets vergelijkbaars te maken maar dan met Dijkstra ernaast om ze te kunnen vergelijken

Dit is wat eruit kwam:



Zoals je kunt zien is A* echt veel efficiënter door de heuristic. Dijkstra moet echt langs bijna alle nodes totdat die uiteindelijk bij het einde komt terwijl A* er veel sneller is. Ze hebben uiteindelijk een vergelijkbaar pad, maar A* is veel efficiënter.

Hoe implementeren we dit in gazebo?

Grid map

De wereld wordt omgezet in een Occupancy Grid Map. Dit is een raster van cellen waarbij elke cel een waarde heeft:

- 0: Vrije ruimte
- 100: Obstakel
- -1: onbekend gebied

Replanning

Terwijl de lidar scant, kan de SLAM kaart veranderen. A* moet dan heel snel een nieuw pad berekenen vanaf de huidige locatie.

Path costs

De robot is natuurlijk niet klein dus hij moet niet te dicht langs de muren gaan, daarom moeten we een inflation layer gebruiken:

- Rondom muren worden de kosten verhoogd
- A* ziet dan dat het pad vlak langs de muren duurder is dan het midden dus dan gaat die wat verder van de muur af erlangs

Heuristische functie

Voor een robot in een gebouw gebruik je meestal een van deze twee:

- Euclidische afstand: de vogelvlucht afstand (kortste lijn). Dit is handig als de robot elke hoek kan draaien
- Manhattan afstand: Alleen horizontale en verticale stappen.

Hoe implementeer je dit in gazebo?

Als je gebruik maakt van Nav2 in Ros2, dan zit A* (of een variant zoals smac planner) al ingebouwd. De robot doet dan het volgende:

1. Global planner: Gebruikt A* om een grove route te plannen van de huidige kamer naar de brand
2. Local planner: Kijkt naar de lidar om kleine obstakels te ontwijken terwijl hij het A* pad probeert te volgen

Lists

Het A* algoritme heeft 2 lists: **Open list**:

- Bevat nodes die nog geevalueerd moeten worden
- Gesorteerd met de laagste waarde van $f(n)$ eerst
- Nieuwe nodes worden toegevoegd als ze worden ontdekt

Closed list:

- Bevat al geevalueerde nodes
- Hierdoor ontwijkt hij al geevalueerde nodes
- Dit wordt gebruikt om het eind pad te vinden

Dit algoritme kiest de node met de laagste waarde van $f(n)$ van de open list, evalueert die, en stopt hem in de closed list totdat die de goal node bereikt of dat die erachter komt dat er geen pad is.

A* pseudocode

```
function A_Star(start, goal):
    // Initialize open and closed lists
    openList = [start]           // Nodes to be evaluated
    closedList = []              // Nodes already evaluated

    // Initialize node properties
    start.g = 0                  // Cost from start to start is 0
    start.h = heuristic(start, goal) // Estimate to goal
    start.f = start.g + start.h    // Total estimated cost
    start.parent = null           // For path reconstruction
    while openList is not empty:
        // Get node with lowest f value - implement using a priority queue
        // for faster retrieval of the best node
        current = node in openList with lowest f value
```

```

// Check if weve reached the goal
if current = goal:
    return reconstruct_path(current)

// Move current node from open to closed list
remove current from openList
add current to closedList

// Check all neighboring nodes
for each neighbor of current:
    if neighbor in closedList:
        continue // Skip already evaluated nodes

// Calculate tentative g score
tentative_g = current.g + distance(current, neighbor)

if neighbor not in openList:
    add neighbor to openList
else if tentative_g >= neighbor.g:
    continue // This path is not better

// This path is the best so far
neighbor.parent = current
neighbor.g = tentative_g
neighbor.h = heuristic(neighbor, goal)
neighbor.f = neighbor.g + neighbor.h

return failure // No path exists
function reconstruct_path(current):
    path = []
    while current is not null:
        add current to beginning of path
        current = current.parent
    return path

```

Bron

Dit heb ik gevonden op een site waar ik ook meer van mijn info vandaag heb
[bron](#)